

Recordatorio

En las iteraciones anteriores se implementó:

- **Iteración 01:** programa en consola (CLI) dentro de un único archivo, versión que sirvió entender la mecánica básica de la ruleta.
- **Iteración 02:** ejercicio de uso de ventanas para el módulo de login y registro en Swing, con usuarios hardcoded y almacenamiento en memoria.
- **Iteración 03:** aplicación del *Principio de Responsabilidad Única* (SRP), separando la lógica de la ruleta en clases.
- **Iteración 04:** incorporación del patrón *Modelo–Vista–Controlador* (MVC) y uso de getters y setters para aplicar encapsulamiento explícitamente.
- **Iteración 05:** implementar relaciones de dependencia y asociación entre clases de manera explícita a nivel de código por medio de un diagrama.
- **Iteración 06:** incorporar proceso *Round-Trip* en Visual Paradigm para el modelado e implementación de un módulo de estadística.

1. Objetivos

1.1 Objetivo General

Refactorizar el paquete «Modelo» para reemplazar flujo por tipo, condicionales y `switch`, a polimorfismo y herencia por medio de la introducción de una jerarquía de apuestas, `ApuestaBase` y subclases concretas.

1.2 Objetivos Específicos

1. En base al diagrama de clases de la Ruleta, sustituir el ENUM por una superclase abstracta llamada `ApuestaBase`, la cual contendrá responsabilidades mínimas como el monto apostado, la etiqueta de la apuesta y el método polimórfico `acierta(int numero, String color)`, junto a las subclases `ApuestaRojo`, `ApuestaNegro`, `ApuestaPar` y `ApuestaImpar`,

las cuales encapsularán su propia lógica de acierto.

2. Modificar `Ruleta.jugar(...)` para que reciba una `ApuestaBase`, calcule una única vez el *número* y el *color*, y delegue la verificación de acierto a la subclase concreta.
3. Reducir complejidad y acoplamiento eliminando condicionales por tipo en el dominio y garantizando que controladores y vistas dependan de abstracciones (superclases o interfaces) en lugar de clases concretas.
4. Incluir manejo de archivos inicial [9], pasando de un enfoque volátil por ejecución a un esquema persistente basado en archivos (lectura y escritura) que conserve los datos entre ejecuciones.

2. Ayudas y Sugerencias

2.1 Polimorfismo en acción

2.1.1. Enfoque anterior

El motivo de cambiar el enfoque de `enum` a una superclase con subclases radica en modularizar el código, ya que este último, basado en el pilar del polimorfismo, lleva el comportamiento específico de cada apuesta a la subclase que corresponde; así, en lugar de usar condicionales para decidir qué tipo de apuesta corresponde, el diseño delega la lógica a cada clase concreta.

```
switch (tipoApuesta) {  
    case ROJO:  acierto = numero != 0 && esRojo(numero); break;  
    case NEGRO: acierto = numero != 0 && !esRojo(numero); break;  
    case PAR:   acierto = numero != 0 && numero % 2 == 0; break;  
    case IMPAR: acierto = numero != 0 && numero % 2 != 0; break;  
}
```

Listing 1: Enfoque previo con condicionales por tipo

2.1.2. Enfoque con polimorfismo [6]

Con este cambio, se declara una variable usando el tipo de la superclase `ApuestaBase`, pero se instancia una subclase concreta según lo que eligió el usuario (`ApuestaRojo`, `ApuestaNegro`, etc).

Nota: Diferencia entre tipo de referencia y tipo real (en memoria)

- **Tipo de referencia:** es el tipo con el que se declara la variable en el código, el cual indica al compilador las operaciones que están permitidas en esa variable.
- **Tipo real (objeto en memoria):** es la clase exacta que se instanció con `new`.

Es esta combinación de tipos que hace posible el polimorfismo en Java, ya que:

- El compilador verifica que se llamen solo a métodos que existen en `ApuestaBase` (porque ese es el tipo de referencia).
- En tiempo de ejecución, la JVM mira el tipo real del objeto y ejecuta la implementación del método (late binding) [8]. No hay que preguntar el tipo con `if` o `switch`: el lenguaje lo resuelve.

```
ApuestaBase apuesta = new ApuestaRojo(200);

// Ruleta solo conoce la abstracción
Resultado r = ruletaCtrl.jugar(apuesta);

public Resultado jugar(ApuestaBase apuesta) {
    int numero      = girar();
    String color     = colorDe(numero);
    boolean acierta = apuesta.acierta(numero, color); // Misma llamada, distinta
    implementación en runtime
}
```

Listing 2: Uso polimórfico en el ejercicio

De este modo, agregar una nueva apuesta implica crear otra subclase con su propia regla, sin modificar `Ruleta` ni tocar los controladores o sumar casos a un `switch`; el sistema crece de forma lineal y predecible.

2.2 Principio de Liskov

En este contexto de modularidad y uso de herencia como herramienta, aparece dentro de los principios SOLID [8], el *Principio de Sustitución de Liskov* el cual establece que si una clase B hereda de A, entonces donde se espera un objeto de tipo A se puede usar uno de tipo B sin romper el contrato.

20GitHub%20-%20IntelliJIDEA.pdf

- [4] [Tutorial] Cómo usar Git y GitHub. <https://github.com/JesusTM/ICC490-1-P00/blob/main/2.Reportes/%5BRT-P00-02%5D%20Uso%20de%20GitHub.pdf>
- [5] Java Round-Trip https://www.visual-paradigm.com/support/documents/vpuserguide/276/381_javaround-tr.html
- [6] [Presentación] Intro Herencia y POO.
<https://campusvirtual.ufro.cl/mod/resource/view.php?id=2037372>
- [7] [Presentación] Intro Herencia (II) y POO.
<https://campusvirtual.ufro.cl/mod/resource/view.php?id=2037375>
- [8] [Infografía] Principios SOLID.
<https://campusvirtual.ufro.cl/mod/resource/view.php?id=2035831>
- [9] [Documentación] Apuntes Manejo de Archivos- Jesús. <https://jessustm.github.io/ICC490-1-P00/1.Apuntes/10.Manipular%20Archivos/>